Performance Analysis, Modelling and Optimisation of Direction-Optimising Breadth-First Search (GAPBS)

Sergiu Alexandru Macra (s5524032)
Cosmin Andrei Stamate (s5600723)
Group 1 — Performance Engineering (WMCS045-05), University of Groningen

Draft, 16 October 2026

Abstract

We study the breadth-first search (BFS) kernel of the GAP Benchmark Suite on two systems: a laptop with a hybrid Intel Core i7-12650H (run under WSL2) and a dual-socket AMD EPYC 7763 node of the Hábrók cluster. With perf, PAPI, gprof, a perf_event-based counter tool of our own and the benchmark's timers, all cross-validated, we identify the bottom-up step and OpenMP synchronisation as the dominant costs. Instrumenting the kernel shows that direction-optimising BFS examines only 4.7 % of the edges of a Kronecker graph with 2^{20} vertices. A traffic model built from these counts, STREAM bandwidth and measured latency and memory-level parallelism predicts single-thread performance within 21 % to 30 %. It shows that the laptop is bandwidth-bound, while the server node is limited by synchronisation and cross-socket effects. Of seven optimisation strategies, three gave significant gains: degree-ordered relabelling (up to 2.8×, also cutting the edges examined by up to 48 %), choosing the thread count and placement (2.1× on the server), and NUMA-aware first touch of the graph (2.1× with threads spread over both sockets). As the analysis predicts, compiler flags, schedules, huge pages and prefetching stayed within run-to-run noise.

1 Introduction

Graph traversal is a computational dwarf whose performance is governed by irregular memory accesses rather than arithmetic. Breadth-first search (BFS) is its canonical kernel: it is the first kernel of Graph500 [6] and of the GAP Benchmark Suite (GAPBS) [1]. Its intensity is far below one operation per byte, so useful models must describe data movement, latency and synchronisation instead of floating-point throughput.

The goal of this project is not the fastest possible BFS. It is a systematic performance engineering process applied to the GAPBS reference implementation: (i) analysing it with several tools whose correctness we check, (ii) explaining the measurements with a semi-analytical model whose parameters come from our own benchmarks, and (iii) choosing, implementing and assessing optimisations that the analysis and the model motivate.

Our contributions are:

- a measurement pipeline that runs every experiment from one script per platform, with compile-time switches that leave the default build identical to upstream (Section 4);
- a cross-validation of five measurement tools, including the observation that gprof cannot attribute time in OpenMP code, and a latent data race in the bottom-up step that appears when the OpenMP schedule is changed (Section 5);
- a traffic and latency model based on instrumented edge counts, which explains the single-thread and laptop results and isolates the synchronisation overhead on a 128-core node (Section 6);

- an evaluation of seven optimisation strategies on two architectures (Section 7).

Our work plan targeted a laptop and a cluster with hardware counters. The HAICGU cluster was not available to us, so we used the RUG Hábrók cluster instead. We also extended the planned compiler-level optimisations with memory-subsystem and parallel-runtime optimisations, because the analysis showed that these dominate.

2 Application: direction-optimising BFS in GAPBS

GAPBS stores graphs in compressed sparse row (CSR) form: an index array of $n+1$ 8-byte pointers and a neighbour array of 4-byte vertex IDs. An undirected graph stores every edge in both directions. Our main input is a Kronecker graph with scale 20 (`kron20`: $n = 2^{20}$ vertices, m = 15,699,691 undirected edges, 136 MB in memory). It is generated once and loaded from a serialised file, because generating and building the graph took 84 % of a naive benchmark run. Larger graphs (`kron22`, `kron24`: 0.5 GB and 2.1 GB) are used to vary the working set.

The kernel (`src/bfs.cc`) implements direction-optimising BFS [2]. It returns the BFS parent of every vertex and uses five performance-relevant routines:

`TDStep` (top-down): every vertex in the frontier queue scans its neighbours and claims unvisited ones with an atomic compare-and-swap on `parent[v]`. The access to `parent[v]` is random. Parallelised over the queue (static schedule).

`BUStep` (bottom-up): every *unvisited* vertex scans its neighbours until it finds one in the frontier bitmap, then stops (early exit). It sweeps all n vertices; neighbour lists are read from their start. Parallelised with `schedule(dynamic,1024)`.

`QueueToBitmap`, `BitmapToQueue` convert the frontier when the direction changes.

`InitParent` initialises `parent` with negated degrees once per search.

`DOBFS` switches to bottom-up when the edges leaving the frontier exceed $1/\alpha$ of the unexplored edges, and back when the frontier shrinks below n/β (defaults α=15, β=18).

Work actually performed. We added an optional edge counter (`-DPE_COUNT_EDGES`). Averaged over 16 searches on `kron20`, a search performs 4.25 top-down and 2.5 bottom-up steps and examines 286,935 edges top-down and 1,202,922 edges bottom-up. That is only 4.7 % of the 31,399,382 directed edges. Bottom-up checks 1,659,885 vertices in total, most of which find a parent in the frontier after reading fewer than one cache line of neighbours. Throughput is conventionally reported in traversed edges per second (TEPS), counting all m input edges (we follow this convention). TEPS therefore overstates the edges actually touched by about 20×, and a model must be based on the real work (Section 6).

3 Computing systems and environment

Table 1 summarises the two systems. Both builds use `-g -fno-omit-frame-pointer` so that call graphs can be sampled; this does not change the generated loop code.

Laptop under WSL2. WSL2 runs Linux in a Hyper-V virtual machine. This has three consequences, which we state here because they bound what the laptop results can show. (i) The hardware performance-monitoring unit is not passed through: all hardware events (cycles, instructions, cache and TLB misses) report *not supported*. Only software events and timer-based sampling are available. (ii) The VM exposes 10 virtual CPUs without hybrid topology, so P- and E-cores cannot be told apart or selected. (iii) Guest memory is translated twice (nested paging), which partly explains the high DRAM latency of 376 ns and adds variance. The laptop's sustainable

Table 1: Systems. Bandwidth and latency were measured by us (Section 6).

	Laptop	Hábrók node (`regular`)
CPU	Intel Core i7-12650H (Alder Lake)	2× AMD EPYC 7763 (Zen 3)
Cores	6 P + 4 E (16 threads); WSL2 exposes 10 vCPUs	2×64, SMT off, 2 NUMA nodes
Caches	L2 1.25 MB/core, L3 24 MB shared	L2 512 KB/core, L3 16×32 MB (one per CCD)
Memory visible	7.6 GB (WSL2 share)	503 GB
STREAM triad, all / 1 thread	15.9 GB/s / 12.3 GB/s	327 GB/s / 34.7 GB/s
Load latency at 1 GB	376 ns	99 ns
OS, kernel	Ubuntu 24.04.1 on WSL2, Linux 6.18	AlmaLinux 9.8
Compiler	GCC 13.3, `-O3 -fopenmp`	GCC 11.5, `-O3 -fopenmp`
Transparent huge pages	`madvise` (default)	`always`
Tools	perf 6.18 (software events, sampling), gprof	PAPI 7.1 (built by us), pecount, gprof

bandwidth of 15.9 GB/s is low for this CPU; we use it as measured. We planned to boot a native Linux on the laptop for hardware counters, but this was not possible in our time frame, so counters come from Hábrók.

Hábrók. The cluster provides neither `perf`, `numactl` nor a PAPI module, and `perf_event_paranoid` is 2 (user-space counting only). We therefore built PAPI 7.1 from source in our home directory. We also wrote `pecount` (`pe/bench/pecount.c`, 200 lines), which counts a command's events through `perf_event_open` like `perf stat`. All Hábrók runs used one exclusive node through Slurm. Even so, our counters were scheduled only 14 % to 26 % of the time when several were open, which indicates that other software on the node also uses the counter hardware. We therefore rely on PAPI regions with few events for quantitative counts.

4 Measurement method and tool validation

Instrumentation without changing the baseline. All modifications are compile-time switches collected in `src/pe_opts.h`. Without them, the code is upstream GAPBS; we only split each `#pragma omp parallel for` into `parallel` + `for`, which has the same semantics. Switches select PAPI regions inside every parallel region (the PAPI high-level API counts only the calling thread), edge counting, `schedule(runtime)`, prefetching, huge pages, parallel first touch, relabelling and the α/β thresholds. A build script compiles 30 variants, and *every variant must pass GAPBS' BFS verifier* (`-v`) before it is timed.

Timing protocol. Each configuration is run as 1 discarded process followed by 5 processes of 24 BFS trials from pseudo-random sources (identical sources across variants and machines). The trial time is GAPBS' own timer around one search. Within a process the first trials are slow (Figure 1): on Hábrók the first five trials take 99 ms to 156 ms against about 4 ms afterwards. We therefore drop trials 1–5 of every process, leaving 95 samples per configuration. We report the median and the interquartile range (IQR) in ms per trial, and GTEPS $= m/t_{\text{median}}$. With IQRs between 14 % (Hábrók, kron22) and 56 % (laptop, kron20) of the median, we treat differences smaller than the IQR as not significant.

Validating the tools. A tool is only useful if it measures what we think it measures. Table 2 lists our checks.

A correctness check that found a data race. Verifying every variant paid off. With `schedule(runtime)` and `OMP_SCHEDULE=dynamic,100` or `guided`, *every* search produced a wrong

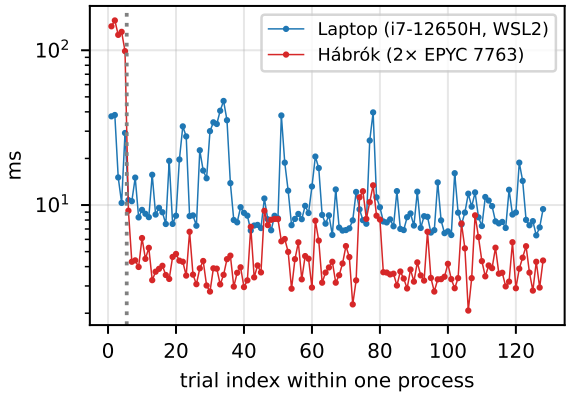


Figure 1: Per-trial time within one process (128 trials, log scale). The slow first trials (Hábrók) and the sporadic outliers (laptop) motivate dropping trials 1–5 and reporting medians.

BFS tree (0/8 verified on both machines), and `static,1000` failed 4/8 times on the laptop and 8/8 on Hábrók. Plain `static` passed, but only by chance: its block boundaries are not guaranteed to fall on word boundaries either. `BUStep` sets bits of the shared `next` bitmap with a non-atomic read-modify-write of a 64-bit word. This is only safe when every thread's block of vertices starts on a word boundary, that is, when the chunk size is a multiple of 64. Upstream's `dynamic,1024` satisfies this implicitly. We restricted the schedule experiments to chunk sizes that are multiples of 64.

5 Performance analysis

Baseline. On `kron20`, unmodified GAPBS with the default thread count needs a median of 8.58 ms per search on the laptop (IQR 4.77 ms, 1.83 GTEPS) and 2.85 ms on Hábrók (IQR 1.43 ms, 5.51 GTEPS). The server node has 12.8× more cores and 20× more bandwidth, yet it is only 3× faster. Explaining this gap is the main question of the analysis.

Hotspots. Table 3 shows where the laptop spends its time, from perf's timer-based sampling (46,000 samples, 64 searches, all threads). Half of all CPU time is threads waiting in the OpenMP runtime. A search consists of about ten short parallel regions (Section 2), each ending in a barrier, and the work inside them is irregular: Kronecker graphs have a heavy-tailed degree distribution, and in bottom-up the cost per vertex depends on how soon a frontier neighbour is found. Among the useful work, `BUStep` dominates by an order of magnitude. Instruction-level annotation of `BUStep` (`pe/results/wsl/perf_annotate_all.txt`) puts 39 % of its samples on the instruction right after the load of the first neighbour ID. Timer sampling skids to the instruction that consumes a long-latency load, so this is the cache miss on the start of each neighbour list. A further 11 % fall on the bitmap bit-test, and about 13 % on loads of the CSR index. The kernel is dominated by memory accesses, not arithmetic.

Hardware counters per kernel (Hábrók). PAPI regions give the per-kernel counters that sampling on the laptop cannot (Table 4, one thread, so every region is fully counted). `BUStep` takes 78 % of the cycles even without any synchronisation, and both steps run at an IPC around 1 on a core that can sustain several instructions per cycle. In `TDStep`, 75 % of the examined edges miss, as expected for random `parent[v]` accesses. In `BUStep` only a third of the checked vertices miss: consecutive vertices' neighbour lists are adjacent in the CSR array, so the hardware prefetcher hides many of them. The conversion kernels are streaming bit operations with high IPC and negligible cost. The data TLB is not a factor on Hábrók: with transparent huge pages always on, a search causes about 4,200 dTLB misses against roughly 2 million random line accesses. Counter runs with

Table 2: Cross-validation of measurement tools.

Check	Result	Conclusion
Timer resolution	`CLOCK_MONOTONIC`: 1 ns; call overhead 34 ns to 36 ns	negligible vs. 1.4 ms+ trials
Wall time, 3 tools (laptop, 64 trials)	GAPBS read+trials 0.679 s; perf `duration_time` 0.717 s; `/usr/bin/time` 0.69 s	agree; 38 ms = process start-up and allocation
CPU time, 2 tools	perf task-clock 6.00 s; `time` user+sys 6.05 s	agree within 1 %
pecount vs. perf	page faults 36,422 vs. 36,423	pecount reproduces perf
gprof	100 % of samples attributed to `frame_dummy`	**unusable** for OpenMP (samples only the main thread; outlined regions unresolved)
perf sampling	BUStep, InitParent, TDStep resolved; libgomp unresolved but attributed by DSO	reliable
PAPI vs. pecount (Hábrók, 1 thread, 64 searches)	sum of the 5 PAPI kernel regions / pecount whole-BFS delta: cache misses 99.1 %, references 97.2 %, cycles 95.0 %, instructions 90.3 %	agree; remainder = code outside the regions (final pass, serial bitmap resets)
Known answer: ALU chain (PAPI)	5.000 instructions and 4.000 cycles per iteration	= 5-instruction loop; critical path `imul` (3) + `add` (1)
Known answer: pointer chase 1 GB (PAPI)	1.0007 cache misses per step	1 miss per random load, as expected
Known answers via pecount	ALU 4.99 instr./iteration; chase 2.4 misses/step	counters were multiplexed (running 14 % to 26 % of the time); scaling is wrong across phases ⇒ use PAPI regions
Timers on Hábrók	`/usr/bin/time` not installed	cross-check only on laptop

128 threads include the five slow warm-up trials, which dominate their wall time, so we do not use them quantitatively.

Thread scaling. Figure 2 shows the time per search as a function of the number of threads. On the laptop the best time is reached at 8 threads (8.39 ms); going to 10 adds no speed and greatly increases variance (IQR 12.3 ms). On Hábrók, time decreases up to 48–64 threads bound to consecutive cores (1.38 ms, 11.4 GTEPS) and *increases* again beyond, to 3.53 ms at 128 threads. 64 threads bound with `OMP_PROC_BIND=close` fill exactly one socket. Adding the second socket adds cross-socket coherence traffic on `parent` and the bitmaps, and more threads per barrier, while each step has less work per thread. Without binding, the best time is 2.45 ms at 48 threads.

Memory hierarchy. Figure 3 shows the dependent-load latency of both machines, measured with a pointer chase over a random cycle. The steps reveal the cache sizes: on Hábrók latency stays at 13 ns to 15 ns up to 32 MB (one CCD's L3) and reaches about 99 ns in DRAM. On the laptop, DRAM latency grows to 376 ns at 1 GB because of TLB misses under nested paging. The working set of a `kron20` search (Section 6) is about 180 MB. It exceeds every cache of the laptop. On Hábrók it exceeds a single CCD's L3 but fits in the 16 L3 slices combined, so much of it is served from remote L3 slices at near-DRAM latency.

6 Performance model

Choice of model. BFS executes almost no floating-point operations, so a classic roofline in FLOP/byte [4] has nothing to place on its horizontal axis. The ECM model needs a steady-state

Table 3: Laptop, perf sampling (task-clock, 5 kHz), `kron20`, 64 trials including graph loading.

Symbol	Samples	Role
OpenMP runtime (`libgomp`, spin-wait at barriers)	50.0 %	idle threads
`BUStep` (outlined parallel region)	35.8 %	bottom-up sweep
`InitParent`	5.7 %	per-search initialisation
`TDStep`	3.1 %	top-down expansion
`BitmapToQueue` + `QueueToBitmap`	2.4 %	frontier conversion
`DOBFS` (final pass) and rest	3.0 %	

Table 4: PAPI per kernel, Hábrók, 1 thread, `kron20`, per search (64 searches). Misses: generic `PERF_COUNT_HW_CACHE_MISSES`.

Kernel	Cycles [M]	Share	IPC	Cache misses [k]	Misses per unit of work
`BUStep`	64.0	77.7 %	0.77	532	0.32 per vertex checked
`InitParent`	11.6	14.0 %	0.89	3.7	—
`TDStep`	3.6	4.4 %	1.01	216	0.75 per edge examined
`BitmapToQueue`	2.4	2.9 %	4.80	0.2	—
`QueueToBitmap`	0.8	0.9 %	1.59	2.5	—

loop with predictable data transfers per iteration, so that the in-core and transfer times can be added up. BFS has data-dependent early exits, atomic updates and a work distribution that changes at every level, so ECM's assumptions do not hold. We therefore use an information-exchange model in the spirit of roofline: we count the bytes that must move and the random accesses that must be waited for, from measured work rather than from the graph size. We bound the time by two machine limits, each measured with our own benchmarks:

$$T_{\mathrm{bw}}(p) = \frac{Q}{\min(p\,B_1,\ B_{\mathrm{all}})}, \qquad T_{\mathrm{lat}}(p) = \frac{N_{\mathrm{rand}}\ L_{\mathrm{DRAM}}}{\mathrm{MLP}\cdot p}, \qquad T_{\mathrm{model}}(p) = \max(T_{\mathrm{bw}}, T_{\mathrm{lat}}),$$

where Q is the data volume of one search, B_1 and B_{all} the STREAM triad bandwidth with one and all threads, N_{rand} the number of random cache-line accesses, L_{DRAM} the dependent-load latency at 1 GB, and MLP the number of misses a core keeps in flight (Little's law). T_{model} is a lower bound: it ignores synchronisation and assumes every access goes to DRAM.

Parameters. Q and N_{rand} follow from the edge counts of Section 2 (Table 5). In bottom-up, every checked vertex costs at least one cache line of its neighbour list (its first neighbours usually suffice). In top-down, every examined edge costs 4 bytes of neighbour ID plus one random line of `parent`. Sweeps over `parent` (4 B), the CSR index (8 B) and the bitmaps are sequential. MLP was measured with up to 32 interleaved pointer chases on one core. On the laptop the time per load falls from 262 ns with one chain to 17 ns with 16 chains, giving MLP = 15.7. This matches the 16 fill buffers of a Golden Cove core. On Hábrók the time per load falls from 101 ns to 4.65 ns with 32 chains, giving MLP = 21.7.

Model vs. measurement. Table 6 and Figure 4 compare the model with the measured times. Three regimes emerge:

1. **Single thread: latency-bound.** On both machines $T_{\mathrm{lat}} > T_{\mathrm{bw}}$: one core cannot keep enough misses in flight to use the bandwidth. On the laptop the measured time is within 21 % of the latency bound. On Hábrók it is 2.8× the bound. The BFS sustains only ≈7.7 outstanding misses ($N_{\mathrm{rand}}L_{\mathrm{DRAM}}/T$) out of the 21.7 the core supports. Each checked vertex is a chain of dependent loads (`parent[u]` → index → neighbour → bitmap word), so the out-of-order window exposes only a few vertices at a time. The laptop's much higher latency

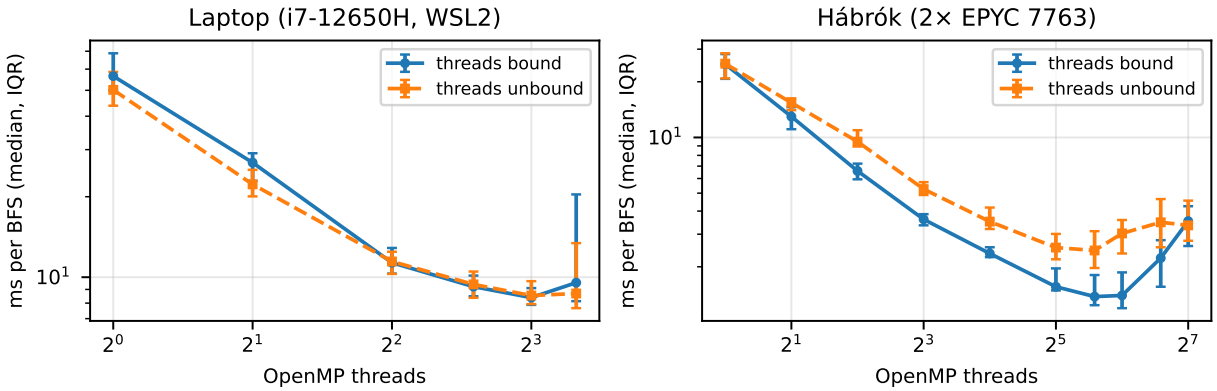


Figure 2: Time per BFS on `kron20` vs. OpenMP threads, with threads bound to consecutive cores (`OMP_PROC_BIND=close`, `OMP_PLACES=cores`) and unbound. Error bars: IQR.

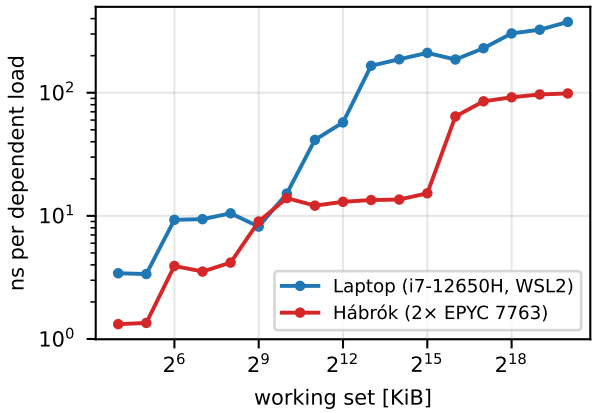


Figure 3: Latency of dependent loads vs. working-set size (one core).

hides this limit better. Including the achievable rather than the peak MLP is the main refinement this model would need.

2. **Laptop, all threads: bandwidth-bound.** With 8 threads the latency bound drops below the bandwidth bound of 11.3 ms. The measured 8.4 ms is even 26 % faster than this bound. Moving all 179 MB from DRAM in 8.4 ms would need 21 GB/s, more than STREAM's 15.9 GB/s, so part of the traffic is served from the 24 MB L3: `parent` (4 MB), the bitmaps and part of the index fit there. The laptop is at its memory-bandwidth limit, which explains why more threads (Figure 2) do not help.

3. **Hábrók, many threads: neither bound.** Both bounds fall to about 0.5 ms, but the best measurement is 1.38 ms (2.5× the bound) and 128 threads take 6.5× the bound. The missing time is synchronisation (about ten barriers per search across up to 128 threads), load imbalance, and cross-socket coherence traffic on shared data. These do not appear in a bandwidth/latency model. This is why the server is only 3× faster than the laptop. It also motivates optimisations that reduce the number of threads or the synchronisation per unit of work rather than the memory traffic.

In terms of attainable performance (m/T_{model}): the DRAM-only bound of the laptop corresponds to 1.4 GTEPS, and with L3 reuse we already reach 1.87 GTEPS, so little can be gained there without reducing Q. On Hábrók the bound corresponds to ≈28 GTEPS for `kron20`, against the 11.4 GTEPS we achieve at best. The server has headroom, but only for optimisations that address synchronisation and placement, or that reduce Q by improving locality.

Table 5: Modelled data volume of one `kron20` search.

Component	MB per search
`InitParent` (index read, `parent` write)	12.6
Bottom-up sweeps, $2.5 \times (12n + n/8)$	31.8
Bottom-up: one neighbour line per checked vertex (1.66M × 64 B)	106.2
Top-down: 0.29M edges × (4 B + 64 B random line)	19.5
Conversions and final pass	8.7
Total Q	**178.8**
Random line accesses N_{rand}	1.95 M

Table 6: Model bounds vs. measured median (bound threads).

Machine	p	T_{bw} [ms]	T_{lat} [ms]	T_{model} [ms]	measured [ms]	ratio
Laptop	1	14.6	46.7	46.7	56.5	1.21
Laptop	8	11.3	5.8	11.3	8.4	0.74
Hábrók	1	5.2	8.9	8.9	24.9	2.81
Hábrók	48	0.55	0.18	0.55	1.38	2.52
Hábrók	128	0.55	0.07	0.55	3.53	6.46

7 Optimisations

Each strategy below is motivated by the analysis or the model. Every variant passes the BFS verifier. Figure 5 summarises speed-ups on `kron20`, and Table 7 gives medians and IQRs. Following Section 4, we call a difference significant only if it exceeds the IQR of both configurations.

Table 7: Median [IQR] in ms per search. Baseline: `-O3`, default schedule and threads. Bold: significant improvement.

Variant	Graph	Laptop	Hábrók
Baseline	kron20	8.58 [4.77]	2.85 [1.43]
O1 `-O0` / `-O2` / `-O3 -march=native`	kron20	20.3 / 8.22 / 9.16	7.03 / 3.07 / 3.43
O2 `static,64` / `dynamic,256` (both loops)	kron20	11.3 / 10.7	2.59 [3.21] / 3.08
O2 threads = 8 (laptop) / 48 (Hábrók), bound	kron20	8.39 [1.23]	**1.38 [0.57]**
O3 THP `never` / `madvise` / patch / `always`	kron20	9.98 / 11.75 / 10.72 / 8.24	– / – / 3.05 / 2.97
O3 huge-page patch	kron22	50.2 [43.2] vs. 71.1 [58.5]	12.83 vs. 12.14
O4 prefetch d=16	kron20	8.15 [3.70]	2.55 [1.57]
O5 degree relabelling	kron20	**3.53 [1.77]**	2.64 [3.13]
O5 degree relabelling	kron22	**25.1 [12.8]** vs. 71.1	**6.67 [1.35]** vs. 12.14 [1.66]
O5 degree relabelling	kron24	–	33.7 [17.3] vs. 38.2 [5.7]
O6 base vs. first touch, spread binding	kron20	–	4.00 [1.17] vs. **1.94 [1.18]**
O6 base vs. first touch, spread binding	kron22	–	12.33 [1.21] vs. **10.96 [1.35]**
O7 α=15, β=9 / α=5, β=9	kron20	8.17 / 37.8	2.42 [1.34] / 2.76

7.1 O1: Compiler optimisation and vectorisation

Rationale. This is the cheapest lever, and it is in our work plan. *Result.* `-O0` is 2.4× slower than `-O3` on both machines; `-O1` recovers most of it; `-O2` and `-O3` are indistinguishable within the IQR. `-march=native` is not faster (9.16 ms vs. 8.01 ms on the laptop, 3.43 ms vs. 2.75 ms on Hábrók, both within the IQR). The GCC vectorisation report explains why: no hot loop is vectorised. The `BUStep` inner loop is rejected because of *control flow in loop* (the early `break`), its outer loop because of *multiple nested loops*, and `TDStep` because the atomic compare-and-swap *clobbers memory*. Vectorising BFS would need a different algorithm formulation. The model also shows the bottleneck is in the memory system, so the instruction count matters little once `-O1` has removed

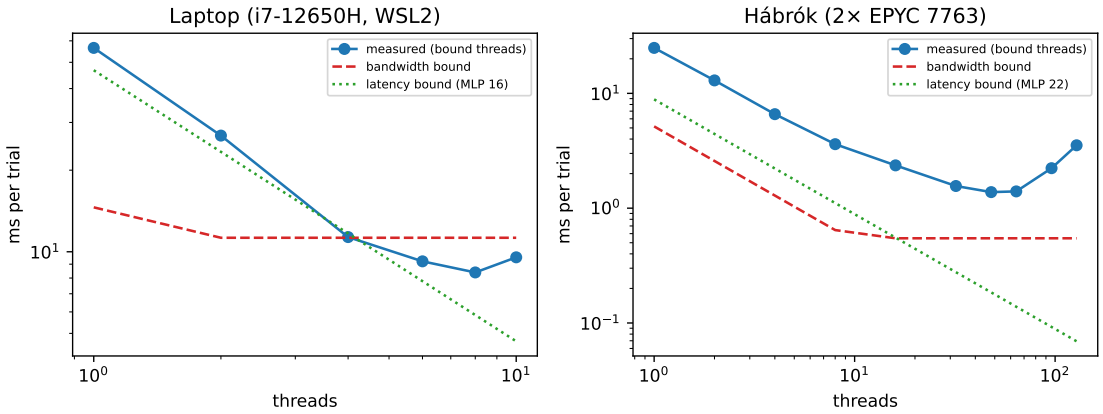


Figure 4: Measured time per search (bound threads) with the bandwidth and latency bounds of the model.

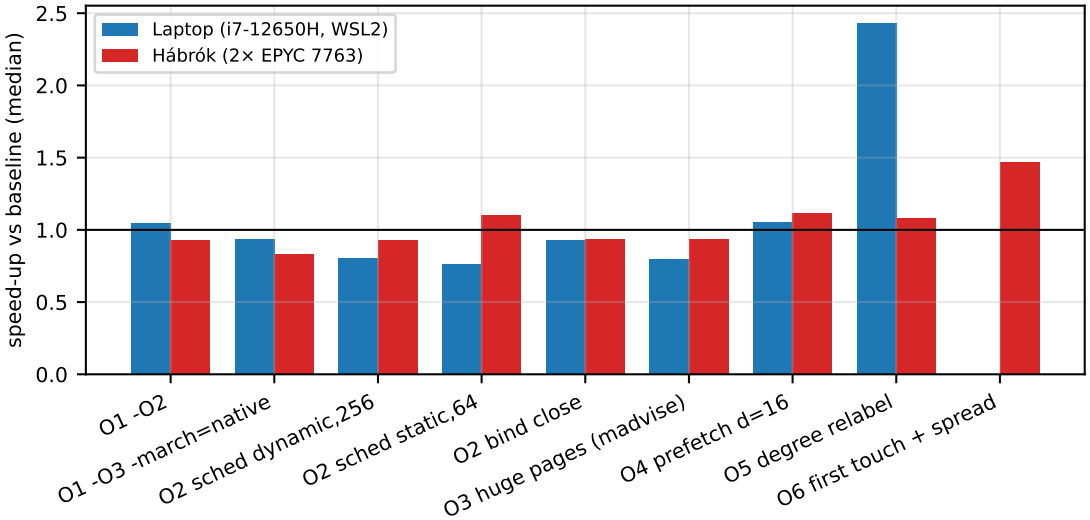


Figure 5: Speed-up of selected variants over the baseline (`kron20`, all threads, ratio of medians).

the gross overhead.

7.2 O2: OpenMP scheduling, binding and thread count

Rationale. Half of the samples are spent in the OpenMP runtime (Table 3), and on Hábrók the model leaves a factor 2.5–6.5 unexplained (Section 6). *Change.* `schedule(runtime)` for both step loops, swept over chunk sizes that are multiples of 64 (Section 4); thread binding; thread count. *Result.* No schedule beats the upstream combination (static top-down, `dynamic,1024` bottom-up). Applying one schedule to both loops is worse on the laptop, and Hábrók's best (`static,64`, 2.59 ms) is within the IQR. Binding alone is not significant. Choosing the thread count is significant: on Hábrók, 48 threads bound to consecutive cores (1.38 ms) are **2.1× faster** than the default 128 unbound threads. They stay on one socket, avoid cross-socket coherence traffic, and pay for smaller barriers. On the laptop the optimum is 8 of 10 vCPUs, which is not significant.

7.3 O3: Huge pages

Rationale. The latency curve (Figure 3) rises beyond the TLB reach, and `BUStep` and `TDStep` access a 136 MB CSR at random. *Change.* `madvise(MADV_HUGEPAGE)` on the CSR arrays and `pvectors` before first touch; we confirmed with `/proc/self/smaps_rollup` that 127 MB of the process then uses 2 MB pages, against 0 without the patch. *Result.* On `kron20` all four laptop configurations lie

within each other's IQR. On `kron22` the patch reduces the laptop median by 29 %, but with IQRs above 40 ms this is not significant. Hábrók already runs with THP `always`, so the baseline uses huge pages and the patch changes nothing: 4200 vs. 4900 dTLB misses per search, both negligible against about 2 million random accesses. On the laptop, where TLB misses inflate DRAM latency (Figure 3), the expected benefit is masked by WSL2's variance.

7.4 O4: Software prefetching in top-down

Rationale. `TDStep`'s random `parent[v]` accesses are latency-bound, and prefetching d neighbours ahead should increase memory-level parallelism. *Result.* No distance is significant on either machine (best: d=16 on Hábrók, −11 %, within the IQR). The analysis predicts this. Top-down accounts for only 3 % of the samples and 11 % of the modelled traffic, so even perfect prefetching could save a few per cent (Amdahl's law). The bottom-up step, where the time goes, already issues independent loads for consecutive vertices, which the out-of-order core overlaps.

7.5 O5: Degree-ordered relabelling

Rationale. The model says the dominant traffic is one neighbour-list line per vertex checked in bottom-up, plus random accesses to `parent`. Renumbering vertices by decreasing degree (`-DRELABEL`, using GAPBS' `RelabelByDegree`) gives the hubs small, adjacent IDs, which improves locality. Because neighbour lists are sorted, hubs, which are most likely to be in the frontier, also come first in every list. *Result.* This is the most effective optimisation: **2.4**× on the laptop (`kron20`), **2.8**× on the laptop and **1.8**× on Hábrók for `kron22`. Our edge counters show why it works: relabelling reduces the edges examined per search by 35 % (`kron20`) and 48 % (`kron22`), because bottom-up finds a frontier parent sooner. So it reduces both Q and the work. On `kron24` the gain is not significant (1.13×, IQR 17 ms). *Caveats.* The one-off relabelling (0.46 s for `kron20`) is excluded from the search time and pays off after about 90 laptop searches. Relabelled graphs have different vertex IDs, so the pseudo-random sources differ from the baseline's.

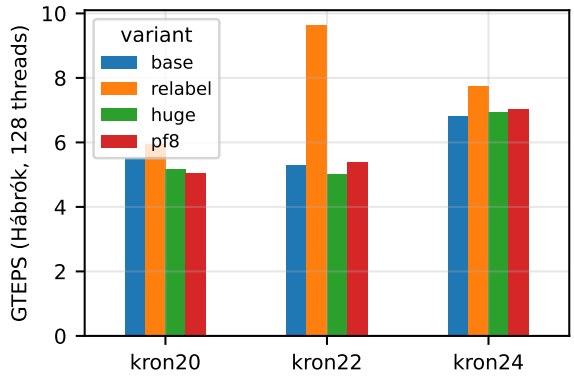


Figure 6: GTEPS on Hábrók for growing graphs: relabelling helps most on `kron22`; huge pages and prefetching do not matter on this node.

7.6 O6: NUMA-aware first touch

Rationale. Loading the graph with a single `read()` touches every page from one thread, so on Hábrók the whole 125 MB edge array lands on one socket. The second socket's threads then read everything remotely, and only one socket's memory controllers are used. `numactl` is not available, so *change*: `-DPAR_FIRST_TOUCH` zeroes the array in parallel before reading. *Result.* Placement matters as soon as threads are spread over both sockets. With `OMP_PROC_BIND=spread`, the baseline slows to 4.00 ms [IQR 1.17] on `kron20`, because half of the threads read everything remotely. With parallel first touch it takes 1.94 ms **[1.18], a 2.1× improvement**, and 1.5× faster than the default

configuration. On `kron22` first touch gains 11 % with spread binding (12.33 → 10.96 ms, significant). Unbound, the gains are smaller and within the IQR, because the OS places threads unpredictably. On `kron24` the difference is not significant; we have not yet determined why the effect disappears for the largest graph.

7.7 O7: Direction-switch thresholds

Rationale. α and β decide how much work runs in the two very different steps. *Result.* A 3×3 sweep shows the defaults are near-optimal. α=5 keeps the search top-down for too long and is 4.4× slower on the laptop. The best settings (α=15, β=36 on the laptop, α=15, β=9 on Hábrók) are within the IQR of the defaults.

8 Discussion and conclusion

What limits BFS. Combining profiles, edge counts and the model gives a consistent picture. A single core is latency-bound: it cannot keep enough misses in flight. A laptop with all cores is bandwidth-bound and already beyond its DRAM bound thanks to L3 reuse. A 128-core server is bound by neither: synchronisation at about ten barriers per search and cross-socket effects dominate. The optimisations confirm this. The significant ones reduce memory traffic or work (relabelling), synchronisation span (fewer, socket-local threads), or remote memory traffic (NUMA-aware first touch). Those that target instruction count (compiler flags, vectorisation) or a minor kernel (prefetching in top-down) were not. This matches the course's message: the justification from analysis and model predicted which strategies could pay off before they were implemented.

Threats to validity. (i) The laptop ran under WSL2: no hardware counters, a hidden hybrid topology and high variance (IQR up to 56 %). We therefore use medians and treat differences within the IQR as insignificant. (ii) The model assumes every byte comes from DRAM; this is why the laptop beats its bound. A cache-aware refinement would use the L3 bandwidth for the 4 MB `parent` array. (iii) Hábrók's counters were multiplexed, and the 128-thread counter runs include warm-up trials, so we use only single-thread counters quantitatively. (iv) Relabelling changes the source vertices. (v) `kron20` is small for a 128-core node. Results on `kron22`/`kron24` show the expected higher GTEPS with growing work per barrier (6.8 GTEPS on `kron24`).

Future work. Hardware counters on the laptop through a native Linux boot; the Apple Silicon comparison planned in our work plan; a NUMA-aware partitioning of the bottom-up sweep; and fusing the short parallel regions of a BFS level to remove barriers. The latter addresses the dominant cost we identified.

References

[1] S. Beamer, K. Asanović, D. Patterson. *The GAP Benchmark Suite*. arXiv:1508.03619, 2015.

[2] S. Beamer, K. Asanović, D. Patterson. *Direction-Optimizing Breadth-First Search*. SC'12, 2012.

[3] J. D. McCalpin. *STREAM: Sustainable Memory Bandwidth in High Performance Computers*. `https://www.cs.virginia.edu/stream/`.

[4] S. Williams, A. Waterman, D. Patterson. *Roofline: an insightful visual performance model for multicore architectures*. CACM 52(4), 2009.

[5] PAPI: Performance Application Programming Interface, ICL, University of Tennessee. `https://icl.utk.edu/papi/`.

[6] Graph500 Benchmark Specification. `https://graph500.org`.

Code and data

All code, scripts and raw results: `https://github.com/CosminStamateAndrei/gapbs` (directory `pe/`; `pe/README.md` explains how every result is reproduced).